

AI/ML-Based Personalized Diet Plan Generator from Medical Milestone 1

Data Collection, OCR & Preprocessing (COMPLETED)

- Collect structured medical datasets for health analysis
- Accept medical data in CSV and image formats
- Extract numeric and textual data from medical reports
- Implement OCR for scanned medical documents
- Preprocess and clean extracted data for ML readiness

Explanation of Implementation

1. Problem Context

Medical reports contain two major types of information:

- **Structured numeric data** such as age, length of hospital stay, and medical outcomes
- **Unstructured textual data** such as medical conditions, treatments, and doctor notes

Patients often struggle to interpret this data and convert it into actionable diet plans. Generic diet recommendations fail to consider individual medical conditions.

Therefore, **Milestone 1 focuses on extracting, cleaning, and structuring medical data**, which forms the foundation of the AI-based personalized diet recommendation system.

2. CSV Dataset Implementation (Structured Medical Data)

Dataset Used

Medical Patient Records Dataset (CSV)

(Processed locally as `medical_data.csv`)

Note: While the project references the *Pima Indians Diabetes Dataset* for conceptual alignment, the implemented CSV preprocessing is based on the uploaded medical dataset containing patient demographics and clinical metadata.

Why This Dataset

- Contains real-world medical and demographic attributes
- Represents practical healthcare records
- Suitable for preprocessing, feature cleaning, and ML preparation

Key Attributes Processed

- Age
- Gender
- Medical Condition
- Treatment
- Outcome
- Smoking Status
- Admission Type
- Length of Stay

Implementation Steps

1. Load CSV data using Pandas
2. Remove duplicate records
3. Handle missing values in categorical columns
4. Convert numeric columns safely
5. Replace missing numeric values with column mean
6. Prepare clean data for future ML models

Code Snippet (CSV Preprocessing)

```
import pandas as pd
```

```
def load_and_clean_csv(csv_path):
```

```
    df = pd.read_csv(csv_path)
```

```
    df = df.drop_duplicates()
```

```
    df.fillna({
```

```
        "Gender": "Unknown",
```

```
        "Medical_Condition": "Unknown",
```

```
"Smoking_Status": "Unknown",
"Insurance_Type": "Unknown",
"Region": "Unknown",
"Admission_Type": "Unknown"
}, inplace=True)

df["Age"] = pd.to_numeric(df["Age"], errors="coerce")
df["Length_of_Stay"] = pd.to_numeric(df["Length_of_Stay"], errors="coerce")

df["Age"].fillna(df["Age"].mean(), inplace=True)
df["Length_of_Stay"].fillna(df["Length_of_Stay"].mean(), inplace=True)

return df
```

3. Image Dataset Usage (Unstructured Data)

Dataset Used

Bajaj Image Dataset (Kaggle)

<https://www.kaggle.com/datasets/dikshaasinghhh/bajaj>

Purpose

- Simulate scanned medical reports and bills
- Provide image input for OCR testing
- Validate extraction of textual data from images

This aligns with the **Medical Report Input (Image/PDF/Text)** stage of the system architecture.

4. OCR Implementation (Text Extraction from Images)

OCR Toolchain

- **EasyOCR** – text extraction
- **OpenCV** – image loading and preprocessing
- **Python** – processing logic

OCR Workflow

Step 1: Image Input

Medical report images are loaded from the local dataset folder.

```
import cv2
```

```
image = cv2.imread(image_path)
```

```
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

Step 2: OCR Reader Initialization

```
import easyocr
```

```
reader = easyocr.Reader(['en'], gpu=False)
```

Step 3: Text Extraction

```
results = reader.readtext(image)
```

```
raw_text = " ".join([text for _, text, _ in results])
```

Step 4: Text Cleaning

```
import re
```

```
clean_text = re.sub(r'^A-Za-z0-9~\s:.', '', raw_text)
```

```
clean_text = re.sub(r'\s+', ' ', clean_text)
```

OCR Output

- Extracted patient-related text
- Medical bill or report details
- Raw unstructured textual information

The extracted text is saved into a CSV file (ocr_results.csv) for further processing.

5. Relation to System Architecture

Milestone 1 successfully completes the following architecture modules:

-  Medical Report Input (CSV & Image)
-  Data Extraction Module

The remaining modules:

- ML Analysis
- NLP Interpretation
- Diet Plan Generator

will be implemented in future milestones.

Conclusion

Through Milestone 1, I learned how real-world healthcare systems manage both structured and unstructured medical data. I gained hands-on experience in CSV preprocessing, handling missing values, and implementing OCR using EasyOCR and OpenCV. This milestone helped me understand how data extraction and preprocessing form the backbone of AI-based healthcare applications and how these components integrate into a larger ML and NLP-driven personalized diet planning system.

Submitted By

Full Name: Sabiha Anjum